

Estrutura de Dados

Aula 06

João Choma Neto
joao.choma@gmail.com

Chamada está em

- <https://github.com/JoaoChoma/EstruturaDados>

Estruturas Lista Fila Pilha

- Lista, fila e pilha são estruturas de dados utilizadas para armazenar e manipular conjuntos de elementos

Lista

- Uma lista é uma estrutura de dados que permite armazenar e manipular coleções de elementos de forma dinâmica
- Uma lista é uma coleção ordenada de elementos, onde cada elemento pode ser acessado por meio de um índice ou uma chave
- A principal característica de uma lista é que os elementos são organizados em sequência e cada elemento é associado a um índice

Lista

- Imagine que temos uma lista de números inteiros {4, 7, 2, 9}
- Podemos adicionar um elemento 5 na posição 2 da lista, fazendo com que ela se torne {4, 7, 5, 2, 9}
- Podemos também remover o elemento 7 da lista, fazendo com que ela se torne {4, 5, 2, 9}

Lista

- As listas também permitem acessar os elementos por índice o que facilita a busca e manipulação de elementos específicos
- Por exemplo, para acessar o elemento na posição 2 da lista, basta utilizar a expressão `lista.get(2)`

Lista

- Existem diferentes tipos de listas, como:
 - Lista simplesmente encadeada
 - Lista duplamente encadeada
 - Lista circular

Lista simplesmente encadeada

- Nesta implementação, a lista é representada por uma classe **ListaSimplesmenteEncadeada**
- Uma referência para o primeiro nó da lista (início)
- Um contador de tamanho
- Cada nó da lista é representado por uma classe interna **Node**, que contém uma referência para o próximo nó da lista e o valor do nó


```
public class ListaSimplesmenteEncadeada {  
    private Node inicio;  
    private int tamanho;  
  
    private class Node {  
        int valor;  
        Node proximo;  
  
        Node(int valor) {  
            this.valor = valor;  
            this.proximo = null;  
        }  
    }  
}
```

```
public ListaSimplesmenteEncadeada() {  
    this.inicio = null;  
    this.tamanho = 0;  
}
```

```
public boolean estaVazia() {  
    return inicio == null;  
}
```

```
public int tamanho() {  
    return tamanho;  
}
```

Lista simplesmente encadeada

- O método **estaVazia** verifica se a lista está vazia e o método **tamanho** retorna o tamanho atual da lista.

```
public void adicionar(int valor) {  
    Node novoNo = new Node(valor);  
  
    if (inicio == null) {  
        inicio = novoNo;  
    } else {  
        Node atual = inicio;  
        while (atual.proximo != null) {  
            atual = atual.proximo;  
        }  
        atual.proximo = novoNo;  
    }  
  
    tamanho++;  
}
```

Lista simplesmente encadeada

- O método **adicionar** adiciona um novo valor ao final da lista, percorrendo a lista até encontrar o último nó e adicionando o novo nó a seguir
- O **tamanho** é incrementado após a adição

```
public void imprimir() {  
    if (estaVazia()) {  
        System.out.println("Lista vazia");  
        return;  
    }  
  
    Node atual = inicio;  
    while (atual != null) {  
        System.out.print(atual.valor + " ");  
        atual = atual.proximo;  
    }  
    System.out.println();  
}  
}
```

Lista simplesmente encadeada

- O método imprimir percorre a lista e imprime cada valor. Se a lista estiver vazia, uma mensagem é exibida indicando que a lista está vazia

Lista simplesmente encadeada - Exemplo

- Criar uma nova lista
- Adicionar três valores
- Imprimir a lista
- Resultado: "10 20 30"


```
public static void main(String[] args) {  
    ListaSimplesmenteEncadeada lista = new  
    ListaSimplesmenteEncadeada();  
    lista.adicionar(10);  
    lista.adicionar(20);  
    lista.adicionar(30);  
    lista.imprimir(); // saída: 10 20 30  
}
```

Entrega atividade – Lista simples

- <https://forms.gle/LuANwVSWUTC96sxf7>

Lista duplamente encadeada

- Segue abaixo um exemplo de implementação de uma lista duplamente encadeada em Java:

```
public class Node {  
    int data;  
    Node prev;  
    Node next;  
  
    public Node(int data) {  
        this.data = data;  
        this.prev = null;  
        this.next = null;  
    }  
}
```

Lista Duplamente Encadeada

- Nesse exemplo, temos duas classes:
 - **Node** e **DuplamenteEncadeada**
- A classe **Node** representa um nó da lista, contendo um atributo data que armazena o valor do elemento, e dois ponteiros: prev e next, que apontam para os nós anteriores e próximos, respectivamente

```
public class DuplamenteEncadeada {  
    Node cabeca;  
    int tamanho;  
  
    public DuplamenteEncadeada() {  
        this.cabeca = null;  
        this.tamanho = 0;  
    }  
}
```

Lista Duplamente Encadeada

- A classe **DuplamenteEncadeada** é responsável por implementar a lista propriamente dita
- A classe possui um atributo cabeça, que aponta para o primeiro nó da lista, e um atributo tamanho, que armazena o tamanho da lista

```
public void adicionar(int data) {  
    Node novoNode = new Node(data);  
  
    if (cabeca == null) {  
        cabeça = novoNode;  
    } else {  
        Node corrente = cabeca;  
        while (corrente.next != null) {  
            corrente = corrente.next;  
        }  
        corrente.next = novoNode;  
        novoNode.prev = corrente;  
    }  
    tamanho++;  
}
```


Lista Duplamente Encadeada

- O método **adicionar** adiciona um novo elemento no final da lista
- Ele cria um novo nó com o valor passado por parâmetro e o adiciona no final da lista, apontando o ponteiro **next** do último nó existente para o novo nó e o ponteiro **prev** do novo nó para o último nó existente

```
public void remover(int data) {  
    if (cabeca == null) {  
        return;  
    }  
  
    if (cabeca.data == data) {  
        cabeca = cabeca.next;  
        cabeca.prev = null;  
        tamanho--;  
        return;  
    }  
}
```

```
Node corrente = cabeca;
while (corrente != null && corrente.data != data) {
    corrente = corrente.next;
}

if (corrente == null) {
    return;
}

if (corrente.next != null) {
    corrente.next.prev = corrente.prev;
}

corrente.prev.next = corrente.next;
tamanho--;
}
```

Lista Duplamente Encadeada

- O método **remove** remove um elemento da lista
- Ele busca o nó que contém o valor passado por parâmetro e o remove da lista, atualizando os ponteiros dos nós anteriores e próximos, caso necessário

```
public int getTamanho() {  
    return tamanho;  
}
```

```
public void imprimir() {  
    Node corrente = cabeca;  
    while (corrente != null) {  
        System.out.print(corrente.data + " ");  
        corrente = corrente.next;  
    }  
    System.out.println();  
}  
}
```

Lista Duplamente Encadeada

- O método **getTamanho** retorna o tamanho da lista
- E o método **imprimir** imprime os elementos da lista na ordem em que eles foram adicionados

Lista Duplamente Encadeada - Exemplo

- Como usar a classe **DuplamenteEncadeada**
- O código cria uma lista duplamente encadeada, adiciona três elementos na lista (10, 20 e 30), imprime os elementos da lista, obtém o tamanho da lista, remove o elemento de valor 20 da lista, imprime os elementos da lista novamente e obtém o tamanho da lista após a remoção

```
public static void main(String[] args) {  
    // Criando a lista  
    DuplamenteEncadeada lista = new DuplamenteEncadeada();  
  
    // Adicionando elementos na lista  
    lista.adicionar(10);  
    lista.adicionar(20);  
    lista.adicionar(30);  
  
    // Imprimindo os elementos da lista  
    System.out.println("Elementos da lista:");  
    lista.imprimir();  
}
```



```
// Obtendo o tamanho da lista
System.out.println("Tamanho da lista: " + lista.getTamanho());

// Removendo um elemento da lista
lista.remover(20);

// Imprimindo os elementos da lista após a remoção
System.out.println("Elementos da lista após a remoção:");
lista.imprimir();

// Obtendo o tamanho da lista após a remoção
System.out.println("Tamanho da lista após a remoção: " +
lista.getTamanho());
}
```

Entrega atividade – Lista Dupla

- <https://forms.gle/dCy4UVzw7jZ4A9ZXA>

Lista Circular

- Uma lista circular é uma estrutura de dados em que o último elemento da lista aponta para o primeiro elemento, criando assim um ciclo. Dessa forma, a lista pode ser percorrida indefinidamente.
- Em Java, uma lista circular pode ser implementada utilizando uma classe que representa os elementos da lista, contendo um atributo para armazenar o valor e outro para armazenar uma referência para o próximo elemento. Para criar a lista circular, basta fazer a referência do último elemento apontar para o primeiro

```
public class NoCircular {  
    private int valor;  
    private NoCircular proximo;
```

```
    public NoCircular(int valor) {  
        this.valor = valor;  
    }
```

```
    public int getValor() {  
        return valor;  
    }
```

```
    public void setValor(int valor) {  
        this.valor = valor;  
    }
```

```
    public NoCircular getProximo() {  
        return proximo;  
    }
```

```
    public void setProximo(NoCircular  
proximo) {  
        this.proximo = proximo;  
    }  
}
```

Lista Circular

- A classe NoCircular representa cada elemento da lista, contendo um atributo para armazenar o valor e outro para armazenar a referência para o próximo elemento

```
public class ListaCircular {  
    private NoCircular primeiro;  
  
    public void adicionar(int valor) {  
        NoCircular novoNo = new NoCircular(valor);  
  
        if (primeiro == null) {  
            primeiro = novoNo;  
            novoNo.setProximo(primeiro);  
        } else {  
            NoCircular ultimo = obterUltimoNo();  
            novoNo.setProximo(primeiro);  
            ultimo.setProximo(novoNo);  
        }  
    }  
}
```

Lista Circular

- A classe ListaCircular é responsável por gerenciar a lista, contendo um atributo para armazenar o primeiro elemento da lista e métodos para adicionar e imprimir os elementos

Lista Circular

- O método **adicionar** recebe um valor e cria um novo nó com esse valor
- Se a lista estiver vazia, o novo nó é o primeiro elemento da lista e sua referência para o próximo elemento é apontada para ele mesmo
- Caso contrário, o novo nó é adicionado no final da lista, com sua referência para o próximo elemento apontando para o primeiro elemento, e a referência do último elemento é atualizada para apontar para o novo nó

<pre> private NoCircular obterUltimoNo() { NoCircular atual = primeiro; while (atual.getProximo() != primeiro) { atual = atual.getProximo(); } return atual; } </pre>	<pre> public void imprimir() { NoCircular atual = primeiro; do { System.out.print(atual.getValor() + " "); atual = atual.getProximo(); } while (atual != primeiro); System.out.println(); } } </pre>
---	--

Lista Circular

- O método **obterUltimoNo** percorre a lista para encontrar o último elemento, que é aquele cuja referência para o próximo elemento é o primeiro elemento
- O método **imprimir** percorre a lista circular a partir do primeiro elemento, imprimindo o valor de cada nó até que o nó atual seja igual ao primeiro elemento novamente

Entrega atividade – Lista Circular

- <https://forms.gle/WxqiTpDr3arfBe41A>

Fila

- Uma fila é uma estrutura de dados onde os elementos são inseridos no final da fila e removidos do início da fila, seguindo o princípio FIFO (First In, First Out)
- É semelhante a uma fila de pessoas em um supermercado, onde a primeira pessoa a entrar na fila é a primeira a ser atendida pelo caixa

Fila

- Uma estrutura de dados do tipo fila (ou queue, em inglês) é uma coleção ordenada de elementos que segue o princípio FIFO (First-In, First-Out), ou seja, o primeiro elemento a ser inserido na fila é o primeiro a ser removido

Fila

- Uma fila pode ser construída utilizando arrays ou listas encadeadas
- O elemento inserido na fila é adicionado ao final da lista e é chamado de "tail".
- O primeiro elemento da fila, chamado de "**head**" ou **cabeca**, é o elemento que será removido

Fila

- Algumas operações básicas de uma fila incluem:
 - enqueue(): adiciona um elemento ao final da fila
 - dequeue(): remove o primeiro elemento da fila e o retorna
 - isEmpty(): verifica se a fila está vazia
 - isFull(): verifica se a fila está cheia (apenas para implementações com array)

Fila

- Um exemplo de uso de fila seria a implementação de um sistema de impressão em uma empresa. Os trabalhos a serem impressos são adicionados na fila, e a impressora é responsável por retirar o primeiro trabalho da fila e imprimi-lo. Dessa forma, garante-se que os trabalhos serão impressos na ordem em que foram recebidos.


```
public class Fila<T> {  
    private LinkedList<T> elementos = new  
    LinkedList<T>();
```

```
    public void enqueue(T elemento) {  
        elementos.add(elemento);  
    }
```

```
    public T dequeue() {  
        return elementos.removeFirst();  
    }
```

```
    public boolean vazia() {  
        return elementos.size() == 0;  
    }
```

```
    public int tamanho() {  
        return elementos.size();  
    }
```

```
    public T primeiro() {  
        return elementos.getFirst();  
    }
```

```
    public T ultimo() {  
        return elementos.getLast();  
    }  
}
```

Fila

- A classe Fila é genérica, podendo receber qualquer tipo de objeto (representado pela letra T). A implementação da fila utiliza uma lista encadeada (**LinkedList**) para armazenar os elementos
- Os métodos **enqueue** e **dequeue** são utilizados para adicionar e remover elementos da fila, respectivamente

Fila

- O método **vazia** verifica se a fila está vazia, o método **tamanho** retorna o número de elementos na fila, e os métodos **primeiro** e **ultimo** retornam, respectivamente, o primeiro e último elemento da fila sem removê-los

Fila - Exemplo

- Adicionar três elementos na fila
- Exibir o tamanho da fila, o primeiro e o último elementos
- Remover os elementos da fila e exibidos na tela
- Verificar se a fila está vazia

```

public class ExemploFila {
    public static void main(String[] args) {
        Fila<String> fila = new
        Fila<String>();

        fila.enqueue("elemento 1");
        fila.enqueue("elemento 2");
        fila.enqueue("elemento 3");

        System.out.println("Tamanho da
        fila: " + fila.tamanho());

        System.out.println("Primeiro
        elemento: " + fila.primeiro());

        System.out.println("Último
        elemento: " + fila.ultimo());

        System.out.println("Removendo
        elementos...");

        System.out.println("Elemento
        removido: " + fila.desenfileira());

        System.out.println("Elemento
        removido: " + fila.desenfileira());

        System.out.println("Elemento
        removido: " + fila.desenfileira());

        System.out.println("Fila vazia? " +
        fila.vazia());
    }
}

```

Entrega atividade - Fila

- <https://forms.gle/rJDobGRGEc3GRHEL9>

Pilha

- Uma pilha é uma estrutura de dados em que os elementos são adicionados e removidos em uma ordem específica, conhecida como LIFO (Last In, First Out), o que significa que o último elemento adicionado é o primeiro a ser removido

```
public class Pilha {  
    private int tamanhoMax;  
    private int[] dados;  
    private int topo;  
  
    public Pilha(int tamanhoMax) {  
        this.tamanhoMax = tamanhoMax;  
        dados = new int[tamanhoMax];  
        topo = -1;  
    }  
}
```


Pilha

- A classe Pilha contém três atributos:
 - **tamanhoMax**, que indica o tamanho máximo da pilha
 - **dados**, que é o array que armazena os elementos da pilha
 - **topo**, que indica o índice do elemento no topo da pilha

```
public boolean isVazia() {  
    return topo == -1;  
}
```

```
public boolean isCheia() {  
    return topo == tamanhoMax - 1;  
}
```

```
public void push(int valor) {  
    if (isCheia()) {  
        System.out.println("A pilha está cheia!");  
        return;  
    }  
    topo++;  
    dados[topo] = valor;  
}
```

Pilha

- Os métodos **isVazia()** e **isCheia()** são responsáveis por verificar se a pilha está vazia ou cheia, respectivamente
- O método **push()** adiciona um elemento ao topo da pilha. Se a pilha estiver cheia, uma mensagem de erro é exibida.

```
public int pop() {  
    if (isVazia()) {  
        System.out.println("A pilha está vazia!");  
        return -1;  
    }  
    int valor = dados[topo];  
    topo--;  
    return valor;  
}  
}
```

Pilha

- O método **pop()** remove o elemento do topo da pilha e o retorna
- Se a pilha estiver vazia, uma mensagem de erro é exibida e é retornado um valor padrão (-1 neste caso).

Pilha - Exemplo

- Criar uma pilha com capacidade máxima de 5 elementos
- Adicionar três elementos à pilha usando o método `push()`
- Método `pop()` é chamado para remover o elemento do topo da pilha e imprimir o valor
- Mais dois elementos são adicionados à pilha e o método `pop()` é chamado novamente para remover e imprimir o valor do novo elemento no topo da pilha

Pilha - Exemplo

```
public static void main(String[] args) {  
    Pilha pilha = new Pilha(5);  
    pilha.push(10);  
    pilha.push(20);  
    pilha.push(30);  
    System.out.println(pilha.pop()); // saída: 30  
    pilha.push(40);  
    pilha.push(50);  
    System.out.println(pilha.pop()); // saída: 50  
}
```

Entrega atividade - Pilha

- <https://forms.gle/tXYtgDbBrJG59MYM8>

ESTRUTURA E CLASSIFICAÇÃO DE DADOS

Aula 06

João Choma Neto
joao.choma@gmail.com